

ICS 知识重点总结

信息表示与 x86-64 机器级程序 · 整理自目录内全部笔记

1. 信息表示与浮点数

1.1. 位、整数、字节序

一字节为 8 bit。同一位串按无符号数或补码解释会得到不同的值。对 w 位向量 $X = [x_{w-1}, \dots, x_0]$:

$$\text{B2U}(X) = \sum_{i=0}^{w-1} x_i 2^i, \quad \text{B2T}(X) = -x_{w-1} 2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i.$$

无符号范围为 $0 \dots 2^w - 1$ ，补码范围为 $-2^{w-1} \dots 2^{w-1} - 1$ 。固定位宽加减与低位乘积按模 2^w 截断；位串本身不带“有符号”标签。比较、右移和类型转换才必须辨别解释方式。

- $\& \mid \sim$ 是按位运算； $\&\& \mid\mid !$ 是逻辑运算，结果为 0 或 1，且 $\&\& \mid\mid$ 会短路。
- 逻辑右移高位补 0，算术右移高位补符号位；移位量为负或不小于操作数宽度是未定义行为。
- 有符号数与无符号数混算可能转成无符号；`sizeof` 返回 `size_t`。无符号倒序变量从 0 再减 1 会回绕，不能用 `i>=0` 停止。

内存按字节编址，指针存地址。x86-64 通常采用小端序：0x01234567 的四字节对象从低地址起为 67 45 23 01；大端序为 01 23 45 67。字节序只改变多字节对象在内存中的排列。C 字符串由单字节字符和结尾 00 组成；用 `unsigned char *` 可逐字节观察对象。64 位机器的指针通常为 8 字节，`int` 通常仍为 4 字节。

1.2. IEEE 754：先分类，再计算

有限数写作 $V = (-1)^s M 2^E$ ，字段为 `s | exp | frac`。设指数域 k 位、小数域 n 位，偏置 $\text{Bias} = 2^{k-1} - 1$ ， $F = \sum_{i=1}^n f_i 2^{-i}$ 。

exp	frac	类别	数值 / 含义
0	0	零	$(-1)^s 0$
0	$\neq 0$	次正规	$(-1)^s F 2^{1-\text{Bias}}$
中间值	任意	规格化	$(-1)^s (1+F) 2^{\text{Exp}-\text{Bias}}$
全 1	0	无穷	$(-1)^s \infty$
全 1	$\neq 0$	NaN	不是实数值

binary32 为 1/8/23 位，偏置 127、有效精度 24 位；binary64 为 1/11/52 位，偏置 1023、有效精度 53 位。binary32 的最小正次正规数为 2^{-149} 、最小正规格化数为 2^{-126} 、1 后的下一数为 $1 + 2^{-23}$ ；binary64 分别为 2^{-1074} 、 2^{-1022} 、 $1 + 2^{-52}$ 。

默认舍入是“最近、正中取偶”：正好落在相邻数中点时，选择最低保留位为 0 的数。浮点加法先对阶、运算、规格化、舍入；乘法先乘有效数并相加指数，然后规格化、舍入。有限二进制小数只能精确表示 $\frac{m}{2^q}$ ；十进制 0.1 的二进制展开无限循环。舍入使加法和乘法通常不满足结合律，乘法也不保证分配律；`fma(a,b,c)` 仅在最终和处舍入一次。

NaN 参与 `< <= > >= ==` 时结果为假，`!=` 为真；`-0` 与 `+0` 数值相等。五类异常为 `invalid`、`divide-by-zero`、`overflow`、`underflow`、`inexact`。整数转浮点可能舍入；binary32 能精确表示绝对值不超过 2^{24} 的整数。浮点转整数向零截断，NaN、无穷或超出目标范围没有可移植的 C 结果。

2. x86-64：数据、条件码与控制流

2.1. 读指令先确定操作数

C 程序经预处理、编译、汇编和链接形成可执行文件。ISA 规定程序可见行为；阅读汇编需跟踪 `%rip`、通用寄存器、条件码和内存。AT&T 语法是 `op Source, Dest`，结果写回目的操作数。后缀 `b/w/l/q` 对应 1/2/4/8 字节；写入 `%eax` 会清零 `%rax` 高 32 位，写入较窄的 `%ax`、`%al` 则不会。

操作数 / 指令	读法	易错点
<code>\$7 / %rax</code>	立即数 7 / 寄存器的值	<code>\$7</code> 不是地址
<code>(%rax)</code>	以 <code>%rax</code> 为地址的内存内容	与 <code>%rax</code> 不同
<code>D(Rb, Ri, S)</code>	$M[R_b + SR_i + D]$	$S \in \{1, 2, 4, 8\}$
<code>leaq D(...), R</code>	$R \rightarrow R_b + SR_i + D$	只计算，不访存

`movq` 可在立即数、寄存器、内存之间传送 8 字节，但通常不能直接“内存到内存”，须经寄存器中转。例：`8(%rdi,%rsi,4)` 的有效地址是 $R[\text{rdi}] + 4R[\text{rsi}] + 8$ ；`movl 8(%rdi,%rsi,4),%eax` 读该地址的 4 字节，`leaq 8(%rdi,%rsi,4),%rax` 则把地址本身写入 `%rax`。`leaq (%rdi,%rdi,2),%rax` 因而可以计算 $3x$ ，且不更改条件码。

2.2. 算术指令与条件码

`addq S,D` 得 $D + S$ ；`subq S,D` 得 $D - S$ ；`imulq S,D` 留乘积低 64 位。`sal/shl` 左移，`sar` 算术右移，`shr` 逻辑右移。四个常用条件码：`CF` 表示无符号进位/借位，`ZF` 表示结果为零，`SF` 是结果符号位，`OF` 表示补码有符号溢出。比如两个同号数相加得到异号结果时，`OF=1`；`CF` 与 `OF` 不能混用。

`cmpq Src, Dest` 按 $\text{Dest} - \text{Src}$ 设置条件码，不保存差值；`testq Src, Dest` 按 $\text{Dest} \wedge \text{Src}$ 设置条件码，不保存与值。例：`cmpq %rsi,%rdi` 后的 `jg` 判断 `%rdi > %rsi`（有符号），`ja` 判断相同位串按无符号解释时 `%rdi > %rsi`。`je/jne` 看 `ZF`；`jg/jge/jl/jle` 用于有符号关系；`ja/jae/jb/jbe` 用于无符号关系。

`setX` 把条件写成低字节 0/1，其他字节保持原样；常需 `movzbl %al,%eax` 零扩展。`jX` 改变 `%rip`；`cmovX` 选择值而不跳转。条件传送通常先算好两边的候选值，因此两边都必须便宜、安全且没有副作用。

读分支实例： 设 `%rdi=x`、`%rsi=y`，下列代码等价于 `return x>y ? x-y : y-x`；。先按 `x-y` 解读 `cmp`，再看 `jle` 何时进入 `else`。

```
cmpq %rsi, %rdi
jle .Lelse
movq %rdi, %rax
subq %rsi, %rax
ret
.Lelse:
movq %rsi, %rax
subq %rdi, %rax
ret
```

2.3. 从跳转恢复 C 控制结构

C 结构	汇编线索
<code>if / else</code>	条件跳转到 <code>else</code> ， <code>then</code> 结束后无条件跳过 <code>else</code> ；共享标签是汇合点。
<code>do-while</code>	循环体先执行，尾部测试并向后跳，至少迭代一次。
<code>while</code>	入口跳到测试，或先入口检查再采用尾部回跳。
<code>for</code>	把 <code>Init; Test; Body; Update</code> 的先后顺序还原为等价的 <code>while</code> 。
<code>switch</code>	稠密 <code>case</code> 常先检查范围，再按下标读取跳转表并间接跳转。

跳转表索引通常按 8 字节地址缩放；`cmpq $6,%rdi; ja default` 可同时排除大于 6 的数和按无符号解释后很大的负数。多个表项可以指向同一代码块；缺失的 `case` 指向 `default`；`fall-through` 体现为代码块之间继续执行。稀疏 `case` 可能采用多次比较。恢复循环时先找回边、入口检查、循环变量更新，再确认零次还是至少一次执行。

3. 过程调用、栈与反汇编实践

3.1. 调用约定与栈帧

Linux x86-64 System V ABI 把前六个整数/指针参数依次放入 `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, `%r9`, 后续参数由调用者放在栈上；整数/指针返回值在 `%rax`。这些是调用边界的约定，参数寄存器进入函数后可以被改作临时寄存器。

类别	规则及寄存器
caller-saved	<code>%rax</code> , <code>%rdi</code> , <code>%rsi</code> , <code>%rdx</code> , <code>%rcx</code> , <code>%r8</code> – <code>%r11</code> ；调用者若需跨调用保留值，应自行保存。
callee-saved	<code>%rbx</code> , <code>%rbp</code> , <code>%r12</code> – <code>%r15</code> ；被调用者使用前保存，返回前恢复。
栈指针	<code>%rsp</code> 指向栈顶，向低地址增长；返回时须恢复到调用约定要求的状态。

`pushq x` 相当于先 `%rsp -= 8` 再写入；`popq x` 先读后 `%rsp += 8`。`call f` 把下一条指令地址压栈，再跳到 `f`；`ret` 弹出返回地址并跳回。栈帧按需保存返回地址、寄存器和局部数据，不是每个函数都要使用 `%rbp`。被取地址的局部变量通常要放在内存中；递归每层有独立返回地址和局部状态。

例如函数调用前把仍需使用的 `dest` 从 caller-saved 的 `%rdx` 移入 `%rbx`，则本函数自身必须 `pushq %rbx` 并在返回前 `popq %rbx`。这体现“作为调用者保护自己的数据”和“作为被调用者恢复上层数据”两种责任同时存在。

按时间追踪调用：若执行 `call f` 前 `%rsp=0x1000`，且下一指令地址为 `0x400120`，调用后 `%rsp=0x0ff8`，内存 `M[0x0ff8]` 存该返回地址；`f` 的 `ret` 读取它，使 `%rip=0x400120`、`%rsp=0x1000`。被调用者若在中途另行压栈，返回前必须撤销相应栈空间。

3.2. 反汇编的系统读法

- 1) 确认函数边界、参数来源、返回值位置以及栈帧布局。静态偏移不一定等于运行时地址；断点优先写作 `function+offset`。
- 2) 顺着每条指令记录寄存器的当前含义。寄存器可复用，编译器也会合并或删除 C 临时变量，因此目标是恢复等价逻辑。
- 3) 每见 `cmp a,b` 先写出 `b - a`，再区分后续跳转的有符号/无符号语义；每见 `lea` 先判断它在构造指针还是做算术。
- 4) 标出后向跳转、共享代码块与间接跳转；整理成循环、分支或跳转表，最后写出伪代码并用调试器核验。

常用 GDB 命令：`disas f` 看函数，`x/i $pc` 看即将执行的指令，`ni/si` 跨过/进入调用，`p/d $eax` 看 32 位整数返回值，`x/s $rsi` 看字符串，`x/6dw $rsp` 看六个栈中整数，`bt` 看调用链。断点停在目标指令执行之前；修改输入文件后须重新 `run`。

3.3. Bomb Lab：六类模式与通用方法

关型	典型结构	关键观察
字符串	逐字节比较	比较函数返回 0 通常表示相等；检查指针所指字符串。
数列	六整数与循环	从栈槽恢复输入顺序，再推导递推和终止条件。
多路	范围检查与跳转表	列出可达表项，追踪共享算术块。
递归	基例与两次递归	写出准确递推，注意跨调用保存的参数。
查表	字符低四位作索引	低位决定映射；按循环逐项累加并比较。
链表	范围/去重/重连	区分输入序号、变换后位置、节点值与 <code>next</code> 指针。

`sscanf` 的前两个参数是输入串和格式串，其余是输出地址；返回值是成功转换并赋值的字段数。要用调用前的 `lea` 和返回后的栈读取共同确认各字段位置。隐藏关要同时检查输入、成功处理函数以及已完成关卡数；有限状态机可还原成 `next[state][input]`，再搜索同步序列并独立验证。

最后自检：不混淆 `%rax` 与 `(%rax)`、值与地址、`sar` 与 `shr`、`jg` 与 `ja`、`CF` 与 `OF`；也不把单次调试中的固定地址或断点编号当作通用规律。